STUDENT ATTENDANCE AND PERFORMANCE TRACKER

Prepared By: Harshit Pandey and Geetha

Enrollment No:EN12026096961
Enrollment No:EN12027097079

Batch code:DA804S80



Introduction

Managing student attendance and academic performance manually can be time-consuming and prone to errors.

This project provides a database system that helps educational institutions store, manage, and analyze student attendance and performance records efficiently.

The system allows quick access to student information, attendance records, examination results, and performance reports.

Problem Statement

Existing Problems

- Attendance records are difficult to maintain manually.
- Student performance data is scattered across different files.
- Generating reports takes significant time.
- Identifying low-performing students is challenging.
- Maintaining data consistency is difficult.
- Lack of data management.

Proposed Solution

Develop a MYSQL-based database system that stores attendance and performance data in a structured format and generates useful reports automatically.

Objectives

- To maintain accurate student, faculty, course, and department records.
- To record and monitor student attendance for each course.
- To manage course enrollments and examination details.
- To track student performance through marks and grades.
- To identify students with low attendance or poor academic performance.
- To generate reports that support academic monitoring and decision-making.
- To reduce manual record-keeping and improve data accuracy.
- To demonstrate the use of SQL concepts such as tables, relationships, primary keys, foreign keys, joins, and queries.

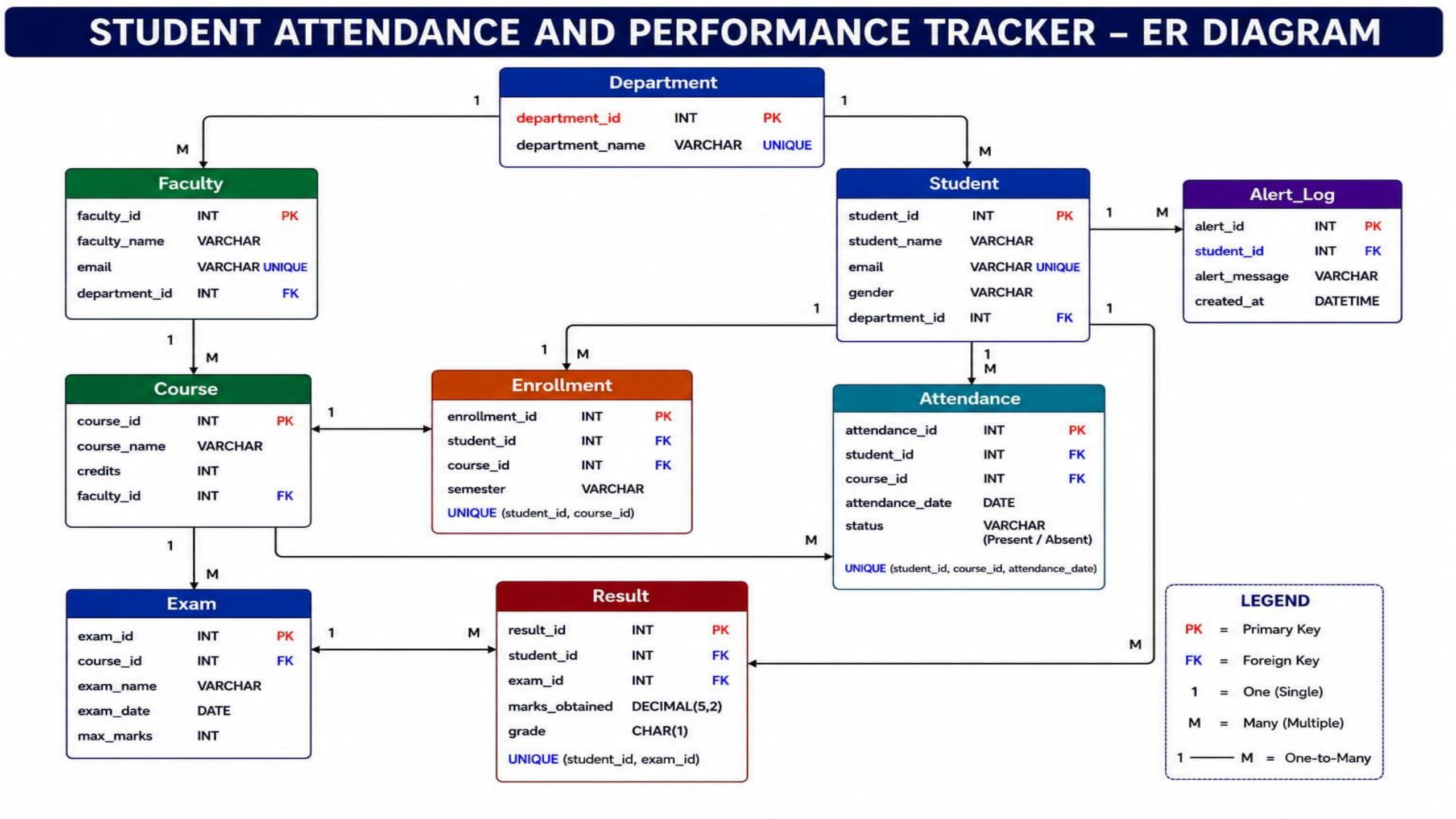
Database Design

The database consists of the following entities:

- Department
- Student
- Faculty
- Course
- Enrollment
- Attendance
- Exam
- Result
- Alert Log

The database is designed in normalized form to reduce redundancy and improve consistency

STUDENT ATTENDANCE AND PERFORMANCE TRACKER – ER DIAGRAM



Database Constraints

Constraints Implemented

To ensure data accuracy and consistency:

- Primary Key
- Foreign Key
- Unique Constraint
- NOT NULL Constraint
- CHECK Constraint

Benefits

- Prevent duplicate records
- Maintain referential integrity
- Improve data quality



SQL Operations Performed

SQL Concepts Used

DDL Commands

- CREATE
- ALTER

DML Commands

- INSERT
- UPDATE
- DELETE

DQL Commands

- SELECT
- WHERE
- ORDER BY

Advanced Concepts

- Joins
- Subqueries
- Aggregate Functions
- Stored Procedures
- Triggers

Important queries

Attendance Analysis

Calculates attendance records for students.

```
SELECT
    student_id,
    ROUND(
        SUM(CASE WHEN status='Present' THEN 1 ELSE 0 END) * 100 / COUNT(*),
        2
    ) AS attendance_percentage
FROM Attendance
GROUP BY student_id;
```

Student Performance Analysis

Displays marks and grades obtained by student

```
SELECT
    s.student_name,
    e.exam_name,
    r.marks_obtained,
    r.grade
FROM Result r
JOIN Student s
ON r.student_id = s.student_id
JOIN Exam e
ON r.exam_id = e.exam_id;
```

Department-wise Student Count

Shows the number of students in each department.

```
SELECT
    d.department_name,
    COUNT(s.student_id) AS total_students
FROM Department d
JOIN Student s
ON d.department_id = s.department_id
GROUP BY d.department_name;
```

Course Enrollment Analysis

```
SELECT
    c.course_name,
    COUNT(e.student_id) AS enrolled_students
FROM Course c
JOIN Enrollment e
ON c.course_id = e.course_id
GROUP BY c.course_name;
```

Joins and Subqueries Joins

Used to combine information from multiple tables.

Examples:

- Student and Department Report
- Student Performance Report
- Attendance Report
- Subqueries

```
SELECT
s.student_name,
d.department_name,
c.course_name,
e.exam_name,
r.marks_obtained,
r.grade
FROM Result r
INNER JOIN Student s
ON r.student_id = s.student_id
INNER JOIN Department d
ON s.department_id = d.department_id
INNER JOIN Exam e
ON r.exam_id = e.exam_id
INNER JOIN Course c
ON e.course_id = c.course_id;
```

Used for advanced analysis.

Examples:

- Students scoring above average marks.
- Highest scoring student.
- Largest department analysis.

```
SELECT
student_id,
marks_obtained
FROM Result
WHERE marks_obtained >
(
SELECT AVG(marks_obtained)
FROM Result
);
```

```
SELECT
s.student_name,
r.marks_obtained
FROM Student s
JOIN Result r
ON s.student_id = r.student_id
WHERE r.marks_obtained =
(
    SELECT MAX(marks_obtained)
    FROM Result
);
```

Stored Procedures

Implemented to automate key academic operations:

- **AddStudent —** Add new student records
- **InsertResult —** Insert marks and generate grades
- **GetStudentPerformance —** Generate student performance reports
- **MarkAttendance —** Record student attendance
- **GetCourseEnrollment —** Display course-wise enrolment count

```sql
-- AddStudent Procedure
DELIMITER //
CREATE PROCEDURE AddStudent(
    IN p_student_name VARCHAR(100),
    IN p_email VARCHAR(100),
    IN p_gender VARCHAR(10),
    IN p_department_id INT
)
BEGIN
    INSERT INTO Student (
        student_name, email, gender, department_id
    )
    VALUES (
        p_student_name, p_email, p_gender, p_department_id
    );
END //
DELIMITER ;

CALL AddStudent('Rahul Sharma','rahul56@gmail.com','Male',1);
```

Triggers

Implemented to enforce data integrity and automate alerts:

- **Automatic Grade Assignment** : Based on marks obtained
- **Marks Validation** : (accepts only values between 0–100)
- **Attendance Alert Generation** : When a student is marked absent
- **Low Attendance Warning** : After repeated absences

```
-- Automatically assigns grade when marks are inserted into Result table.

DELIMITER //
CREATE TRIGGER trg_auto_grade
BEFORE INSERT ON Result
FOR EACH ROW
BEGIN
    IF NEW.marks_obtained >= 90 THEN
        SET NEW.grade = 'A';
    ELSEIF NEW.marks_obtained >= 75 THEN
        SET NEW.grade = 'B';
    ELSEIF NEW.marks_obtained >= 60 THEN
        SET NEW.grade = 'C';
    ELSEIF NEW.marks_obtained >= 40 THEN
        SET NEW.grade = 'D';
    ELSE
        SET NEW.grade = 'F';
    END IF;
END //
DELIMITER ;
```

EVENTS

Event Scheduler Implementation

- Implemented MySQL Event Scheduler for automated attendance monitoring.
- Created a **daily attendance check event** that runs automatically every day.
- Identifies students having **3 or more absences** from attendance records.
- Automatically generates warning entries in the **Alert_Log** table.
- Demonstrates database automation using scheduled tasks.

Benefits

- Automated attendance monitoring.
- Timely warning generation for students with low attendance.
- Reduces manual effort and improves efficiency.
- Ensures consistent execution of attendance checks.

Outcome

- Students with multiple absences are automatically flagged.
- Warning messages are stored in Alert_Log for further action.
- Attendance tracking process becomes fully automated.

```
DROP EVENT IF EXISTS daily_attendance_check;
CREATE EVENT daily_attendance_check
ON SCHEDULE EVERY 1 DAY
STARTS CURRENT_TIMESTAMP
DO
INSERT INTO Alert_Log
(
    student_id,
    alert_message
)
SELECT
    student_id,
    'Attendance Warning: Multiple absences detected'
FROM Attendance
WHERE status = 'Absent'
GROUP BY student_id
HAVING COUNT(*) >= 3;
```

alert_id	student_id	alert_message	created_at
7	1	Event Scheduler Test	2026-06-17 15:29:59
6	1	Event Scheduler Test	2026-06-17 15:29:00
4	8	Performance improvement required	2026-06-12 11:44:05
3	7	Attendance below 75%	2026-06-12 11:44:05
2	4	Attendance below 75%	2026-06-12 11:44:05

Db	Name	Definer	Time zone	Type	Execute at	Interval value	Interval field	Starts	Ends	Status	Originator	character_set_client
student_attendance_performance_tracker	daily_attendance_check	root@localhost	SYSTEM	RECURRING	NULL	1	DAY	2026-06-17 15:30:04	NULL	ENABLED	1	utf8mb4
student_attendance_performance_tracker	test_event	root@localhost	SYSTEM	RECURRING	NULL	1	MINUTE	2026-06-17 15:28:59	NULL	ENABLED	1	utf8mb4

LOCKS

Locking Mechanism Implementation

- Implemented **row-level locking** using MySQL transactions.
- Used **FOR UPDATE** lock on Result and Attendance tables.
- Prevents simultaneous modification of records during processing.
- Ensures consistency while generating performance reports and attendance analysis.
- Implemented **LOCK IN SHARE MODE** for safe data viewing without allowing concurrent updates.

Locks Implemented

Exclusive Lock (FOR UPDATE)

- Result table during performance analysis.
- Attendance table during attendance processing.

Shared Lock (LOCK IN SHARE MODE)

- Result records during report viewing.
- Allows multiple users to read data while preventing modifications.

- **Outcome**
- Prevents conflicting updates.
- Maintains data integrity.
- Supports reliable transaction processing.

```
1  USE student_attendance_performance_tracker;
2  START TRANSACTION;
3  SELECT *
4  FROM Result
5  WHERE result_id = 2
6  FOR UPDATE;
7
8  SELECT CONNECTION_ID();
9  SELECT @@autocommit;
10 SET autocommit = 0;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

result_id	student_id	exam_id	marks_obtained	grade
2	1	2	93.00	B
NULL	NULL	NULL	NULL	NULL

```
9
10 SELECT CONNECTION_ID();
11
12 USE student_attendance_performance_tracker;
13
14 UPDATE Result
15 SET marks_obtained = 90
16 WHERE result_id = 2;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
6	12:58:55	SELECT CONNECTION_ID() LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
7	12:59:02	USE student_attendance_performance_tracker	0 row(s) affected	0.000 sec
8	12:59:12	UPDATE Result SET marks_obtained = 90 WHERE result_id = 2	Running...	?

```
START TRANSACTION;

SELECT *
FROM Result
WHERE student_id = 1
LOCK IN SHARE MODE;
```

```
START TRANSACTION;

SELECT *
FROM Attendance
WHERE student_id = 56
FOR UPDATE;
```

JSON Implementation

JSON was used to store additional student profile information.

Example:

```
{
 "sports":"Cricket",
 "club":"Coding Club",
 "certification":"SQL"
}
```

Functions Used

JSON Object

JSON_EXTRACT()

```
SELECT
    student_id,
    JSON_EXTRACT(profile_data, '$.sports') AS sports
FROM Student_Profile_JSON;
```

profile_id	student_id	profile_data
1	1	{"club": "Coding Club", "hobby": "Reading", "sports": "Cricket", "certification": "Python"}
2	2	{"club": "Music Club", "hobby": "Traveling", "sports": "Football", "certification": "Java"}
3	3	{"club": "AI Club", "hobby": "Chess", "sports": "Badminton", "certification": "SQL"}

Performance Optimization

Query Optimization

Indexes were created to improve query performance.

Indexed Columns

- student_id
- course_id
- Student_name

```
CREATE INDEX idx_student_name ON Student(student_name);

CREATE INDEX idx_course_id ON Course(course_id);

CREATE INDEX idx_result_student ON Result(student_id);

CREATE INDEX idx_attendance_student ON Attendance(student_id);

CREATE INDEX idx_enrollment_student ON Enrollment(student_id);
```

Benefits

- Faster searching
- Improved reporting speed
- Better database performance

Results and Outcomes

The system successfully:

- Tracks student attendance efficiently.
- Stores examination results securely.
- Generates analytical reports.
- Supports academic performance monitoring.
- Maintains data integrity and consistency.

Business Value

Helps institutions make better academic decisions using accurate data.

Conclusion

The Student Attendance and Performance Tracker provides an efficient solution for managing student records, attendance, and academic performance using SQL.

The project demonstrates database design, query writing, automation, optimization, and reporting techniques.

Future Scope

- Web Application Integration
- Real-Time Attendance System
- Dashboard and Data Visualization
- Student Performance Prediction

Questionare

- Q1. What problem does your project solve?
- Q2. Explain your ER Diagram.
- Q3. Why did you normalize the database?
- Q4. Which SQL concepts did you use in this project?
- Q5. Why did you use JOINs and Subqueries?
- Q6. Why did you use Stored Procedures and Triggers?
- Q7. What did you learn from this project?


Thank You